

Graph Valid Path - Disjoint Set Union (DSU)

© Authored by Arpit Raj, LNMIIT Jaipur | August 3, 2026

| C++ Implementation

```

class dsu{
public:

    vector<int> parent;
    vector<int> size;
    dsu(int n){
        parent.resize(n);
        size.resize(n,1);

        for(int i=0;i<n;i++){
            parent[i]=i;
        }
    }

    int findparent(int node){
        if(parent[node]==node){
            return node;
        }

        parent[node]=findparent(parent[node]);
        return parent[node];
    }

    void unionit(int u,int v){
        int pu=findparent(u);
        int pv=findparent(v);

        if(pu==pv){
            return;
        }

        if(size[pu]<size[pv]){
            swap(pu,pv);
        }

        size[pu]+=size[pv];
        parent[pv]=pu;
    }
};

class Solution {
public:
    bool validPath(int n, vector<vector<int>>& edges, int source, int destination) {

```

```

// basically dsu template use kr lenge
// agar source aur destination ka parent same hua to beech me path rhega

dsu ds(n);
for(auto &it:edges){
    int u=it[0];
    int v=it[1];

    ds.unionit(u,v);
}

if(ds.findparent(source)==ds.findparent(destination)){
    return true;
}
else{
    return false;
}
}
};

```

Complexity Comparison

Approach	Time Complexity	Space Complexity
DFS	$O(V + E)$	$O(V + E)$ (Adjacency List + Visited + Recursion Stack)
BFS	$O(V + E)$	$O(V + E)$ (Adjacency List + Visited + Queue)
DSU	$O(E \times \alpha(V)) \approx O(E)$	$O(V)$

Why is DSU slightly better here?

DFS/BFS

First you build the graph:

$O(E)$

DSU

You don't even need an adjacency list.

Then traverse it:

$O(V+E)$

Overall: $O(V+E)$

For every edge:

$\text{union}(u, v)$

Each union is approximately constant time because of:

- Path Compression
- Union by Size

So $O(E \times \alpha(V))$ where $\alpha(V)$ is the Inverse Ackermann Function.

What is $\alpha(V)$?

It's an extremely slowly growing function. For all practical values of V (even billions), $\alpha(V) \leq 4$

So we treat $O(E \times \alpha(V))$ as $O(E)$

Query Scenarios

Single query

Example:

Is source connected to destination?

DFS, BFS, and DSU all work. Difference is negligible.

Multiple queries ★★★

Suppose the question becomes 10^5 queries:

Is u connected to v?

Now DFS/BFS:

Each query $\rightarrow O(V+E)$

Total: $O(Q(V+E))$ **Very slow.**

DSU:

Build once: $O(E)$

Each query: $\text{findParent}(u) == \text{findParent}(v) \approx O(1)$

Total: $O(E + Q)$